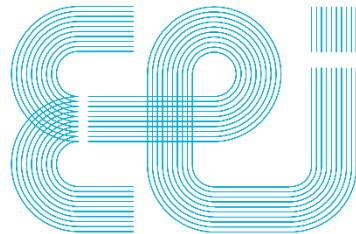


PROGRAMACIÓN TwinCAT

Lab 01

Automatización Industrial
Grado Ingeniería Industrial y Automática



El contenido de esta presentación está basado en los materiales ofrecidos por
Jakob Sagatowski en <https://www.sagatowski.com/>

ÍNDICE

- Breve historia TwinCAT
- SoftPLC
- “Hola Mundo”
- Tipos de Variable
- Tipos de Variable Compuestos
- Punteros y Referencias

RELÉS



Relays by Signalhead at the English Wikipedia licensed under CC BY-SA 3.0

PRIMER PLC

- Desarrollo paralelo:
 - General Motors
 - Allen-Bradley (Rockwell)
- Modicon 084
 - 1968
 - primer PLC
 - fracaso comercial
- Modicon 184
 - 1973
 - éxito ventas

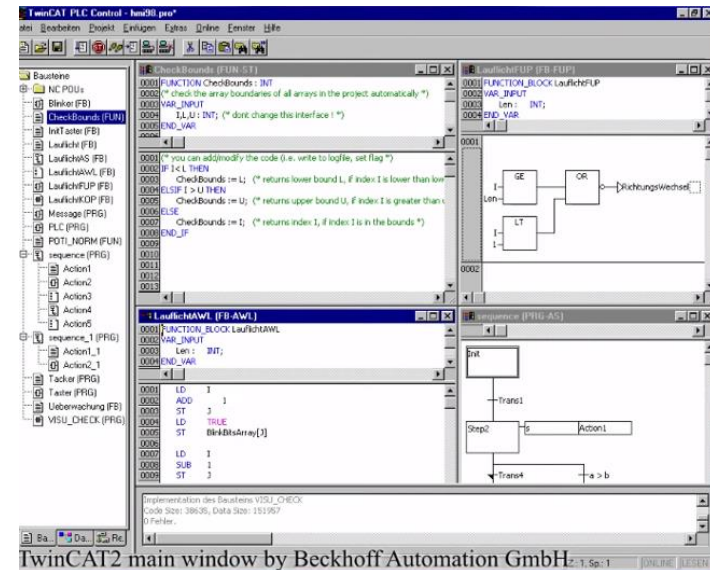


Modicon 184 by AutomationDirect.com



TWINCAT

- Beckhoff Automation (Alemania, 1980)
- 1986, C-4000, primer autómata basado DOS
- TwinCAT = The Windows Control and Automation Technology



TWINCAT3

- 2011
- Virtual Studio para desarrollo (XAE)
- Runtime CODESYS (XAR)
- Licencia desarrollo y ejecución para uso personal gratuita (2 h runtime)

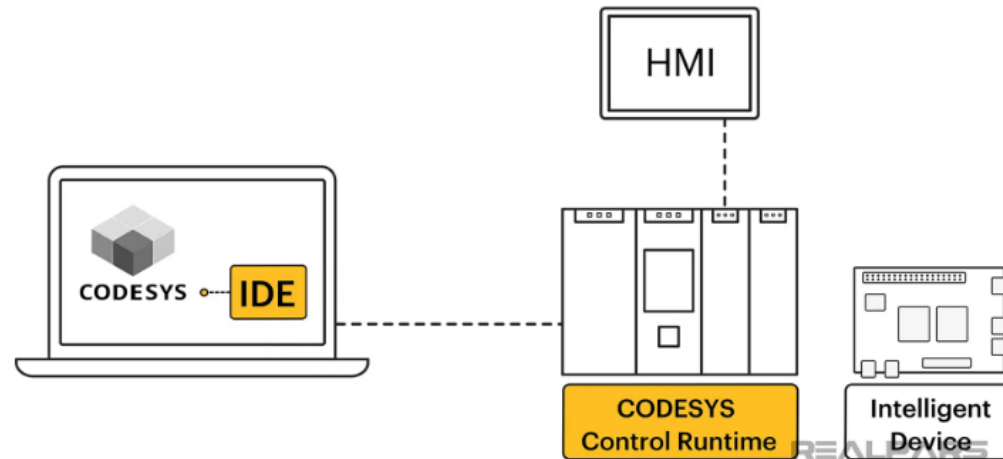


CODESYS

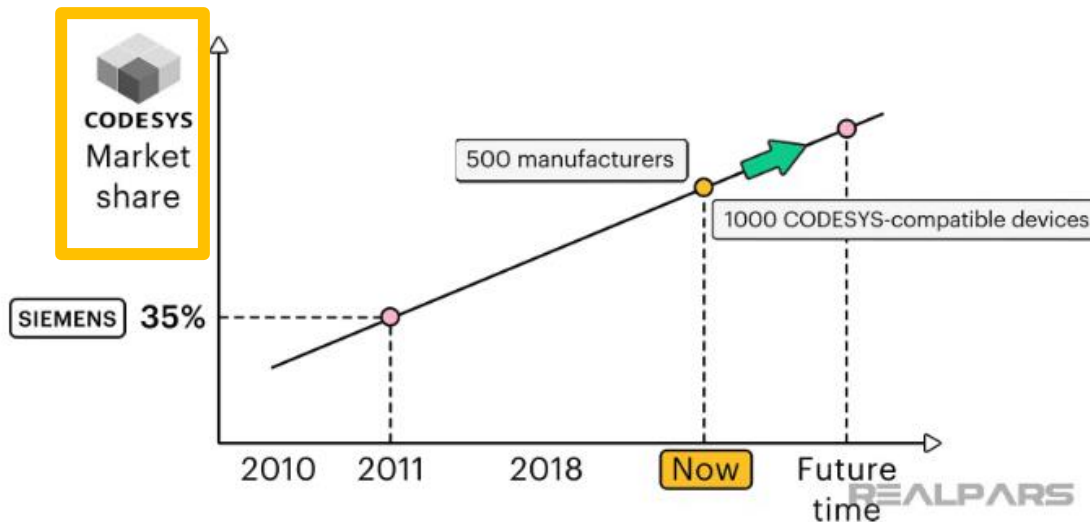
Fuente:

https://www.reddit.com/r/PLC/comments/ejdfqt/codesys_v3_vs_twincat_v3/

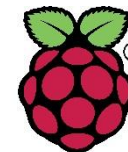
CODESYS



Fuente: <https://www.realpars.com/blog/codesys-basics>



Schneider
Electric



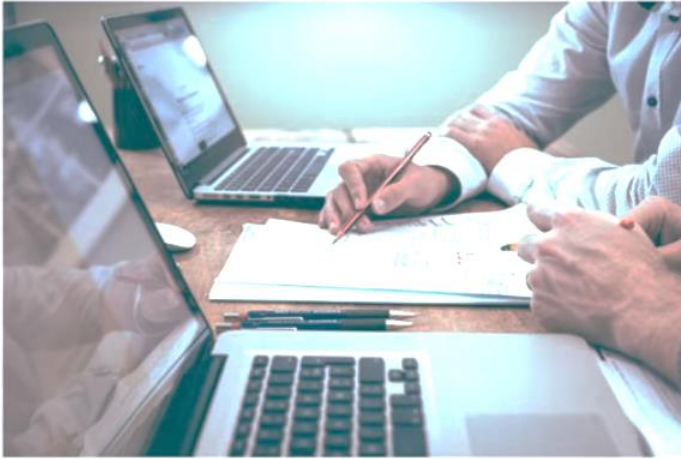
Raspberry Pi



Rockwell
Automation

IT SOFTWARE

”Traditional” IT-software

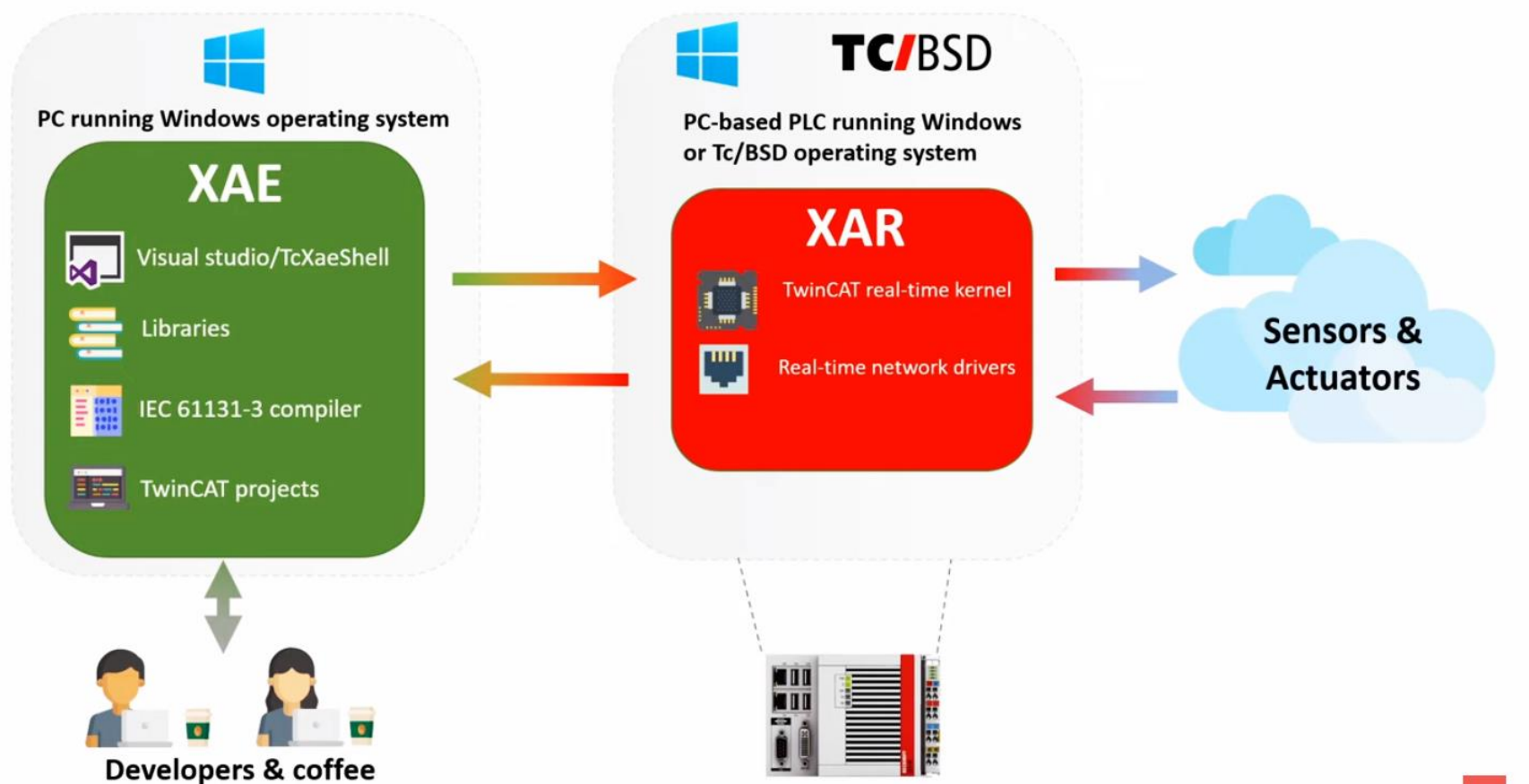


TwinCAT y CODESYS usan técnicas **desarrollo software** en programación PLC y permiten aprendizaje **sin coste.**

Automation software



XAE y XAR



XAE = eXtended Automation Engineering

XAR = eXtended Automation Runtime (no compatible CODESYS)

XAE



Visual Studio 2013

- Community
- Professional
- Enterprise



Visual Studio 2015

- Community
- Professional
- Enterprise



Visual Studio 2017

- Community
- Professional
- Enterprise



Visual Studio 2019

- Community
- Professional
- Enterprise



Any future Visual Studio...

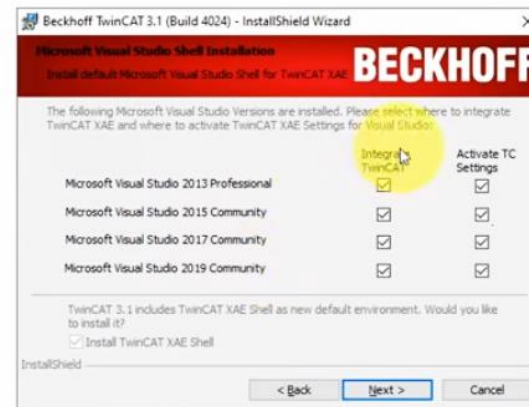
- Community
- Professional
- Enterprise
- XYZ...

TwinCAT® 3 XAE

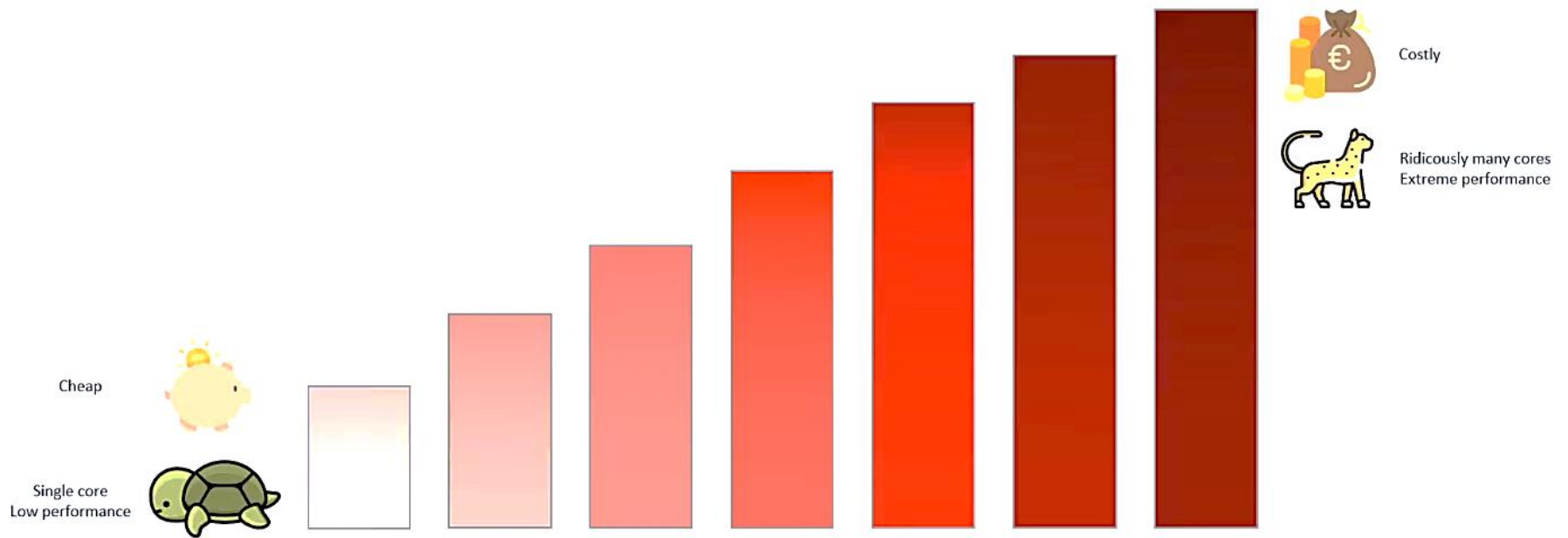


TwinCAT XAE Shell (TcXaeShell)

- Based on Visual Studio
- Included in TwinCAT 3 XAE



MONETIZACIÓN TWINCAT

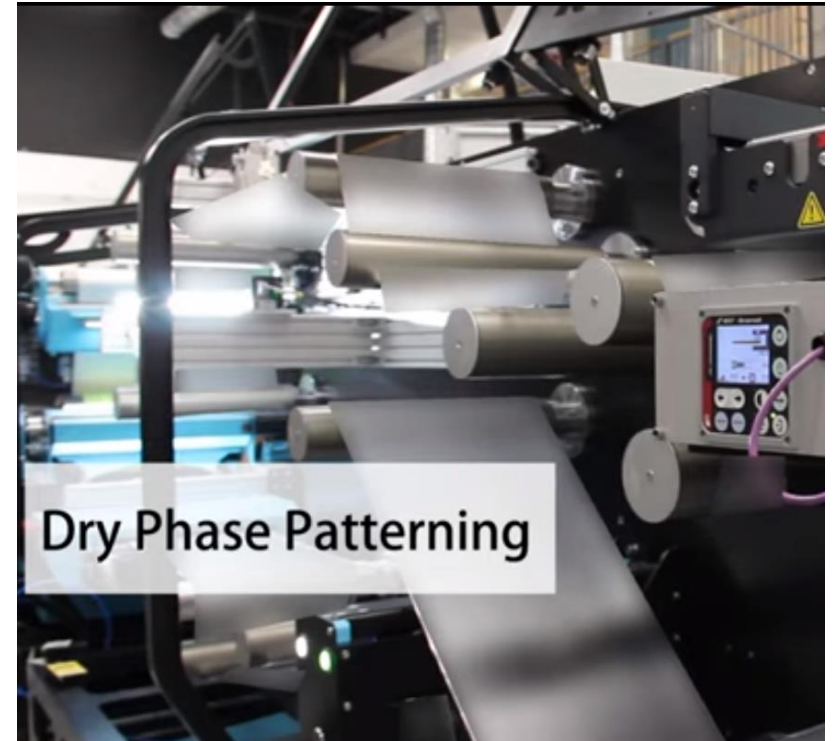


Un PC no-TwinCAT siempre
es asociado con gama alta

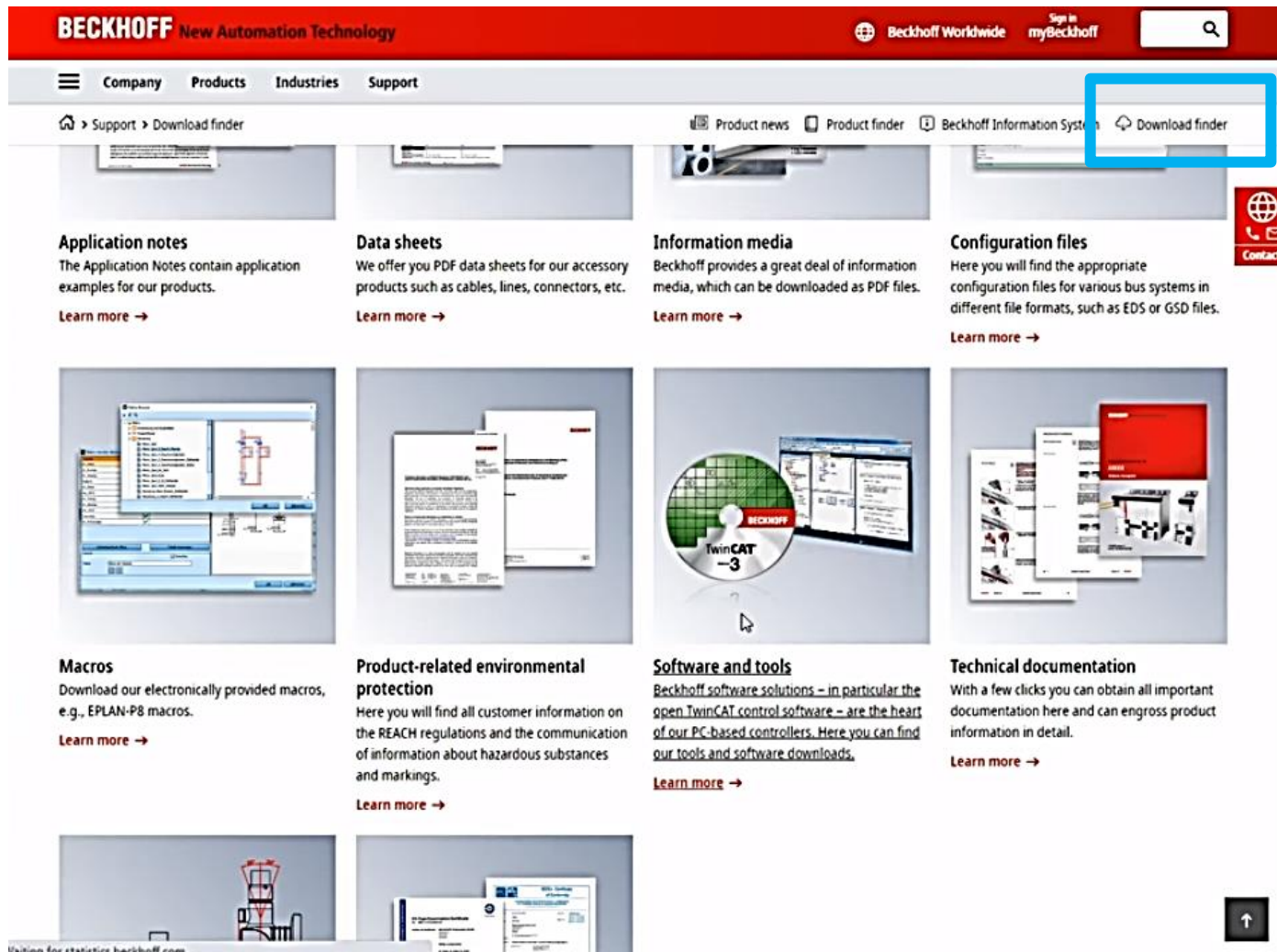
LENGUAJES TWINCAT (IEC 61131-3)

- Sequential Function Charts (SFC)
- Ladder Diagram (LD)
- Function Block Diagram (FBD)
- Instruction List (IL)
- Structured Text (ST)

APLICACIONES TWINCAT



DESCARGA



Download Finder

Fuente: <https://www.beckhoff.com/es-es/support/download-finder/>

VIRTUAL BOX

- Evitar sobrecargar uso diario PC personal
- Recomendable máquina virtual Win10 o Win11
- 8 GB RAM, 4 CPU



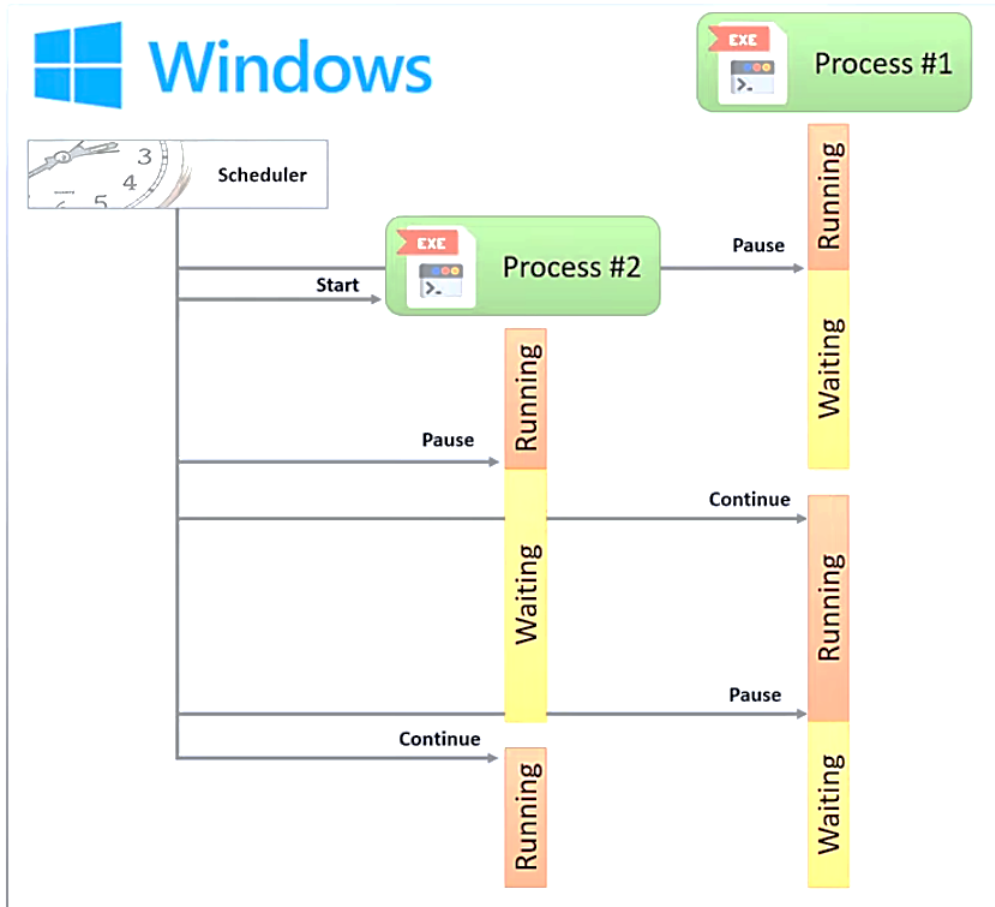
ÍNDICE

- Breve historia TwinCAT
- **SoftPLC**
- “Hola Mundo”
- Tipos de Variable
- Tipos de Variable Compuestos
- Punteros y Referencias

CAPAS SISTEMA OPERATIVO



POLÍTICAS USO CPU



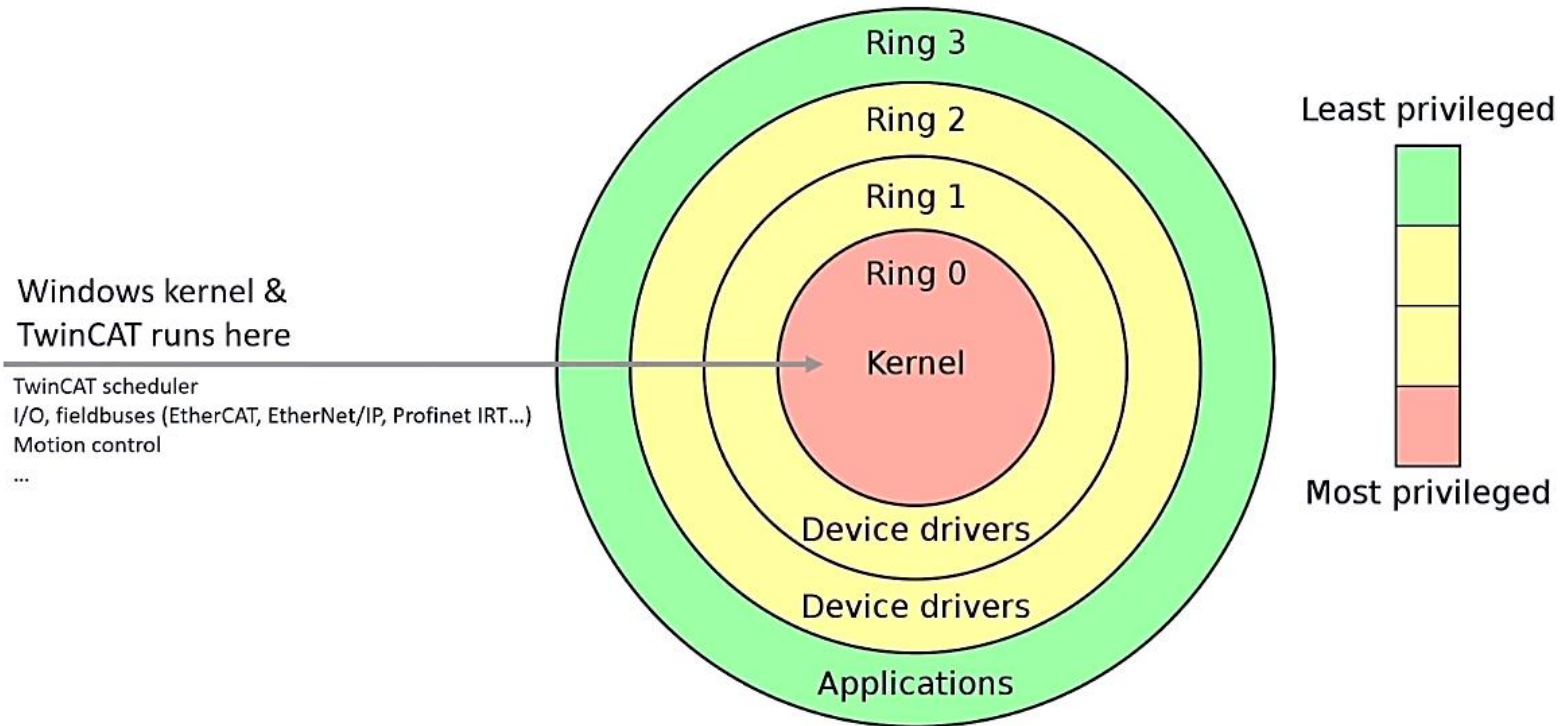
Scheduling policies

- First in, first out (FIFO)
- Shortest job first
- Round-robin
- Fixed priority pre-emptive scheduling
- Manual scheduling
- Priority scheduling
- ...

Windows 10

Multilevel feedback queue

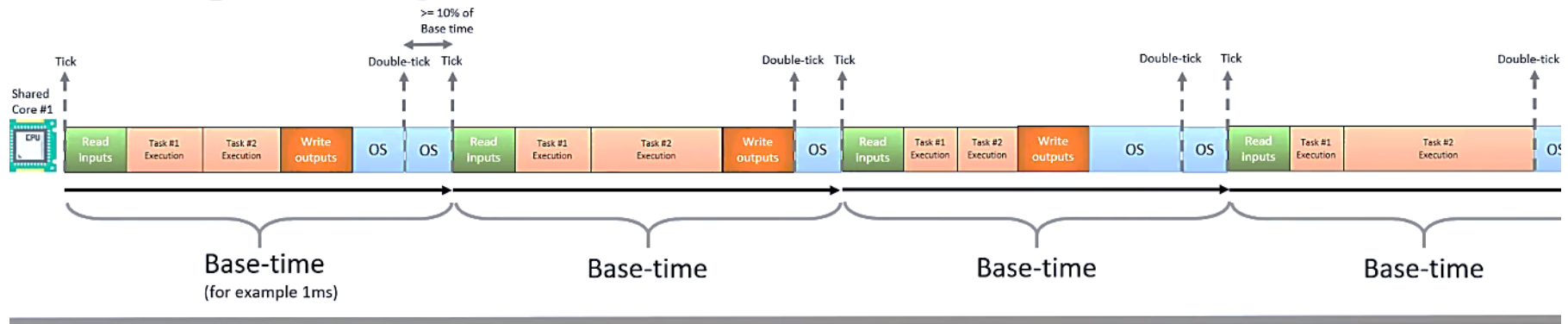
PRIVILEGIOS USO CPU



Attribution: Priv Rings, Hertzprung at English Wikipedia, licensed under CC BY 2.0

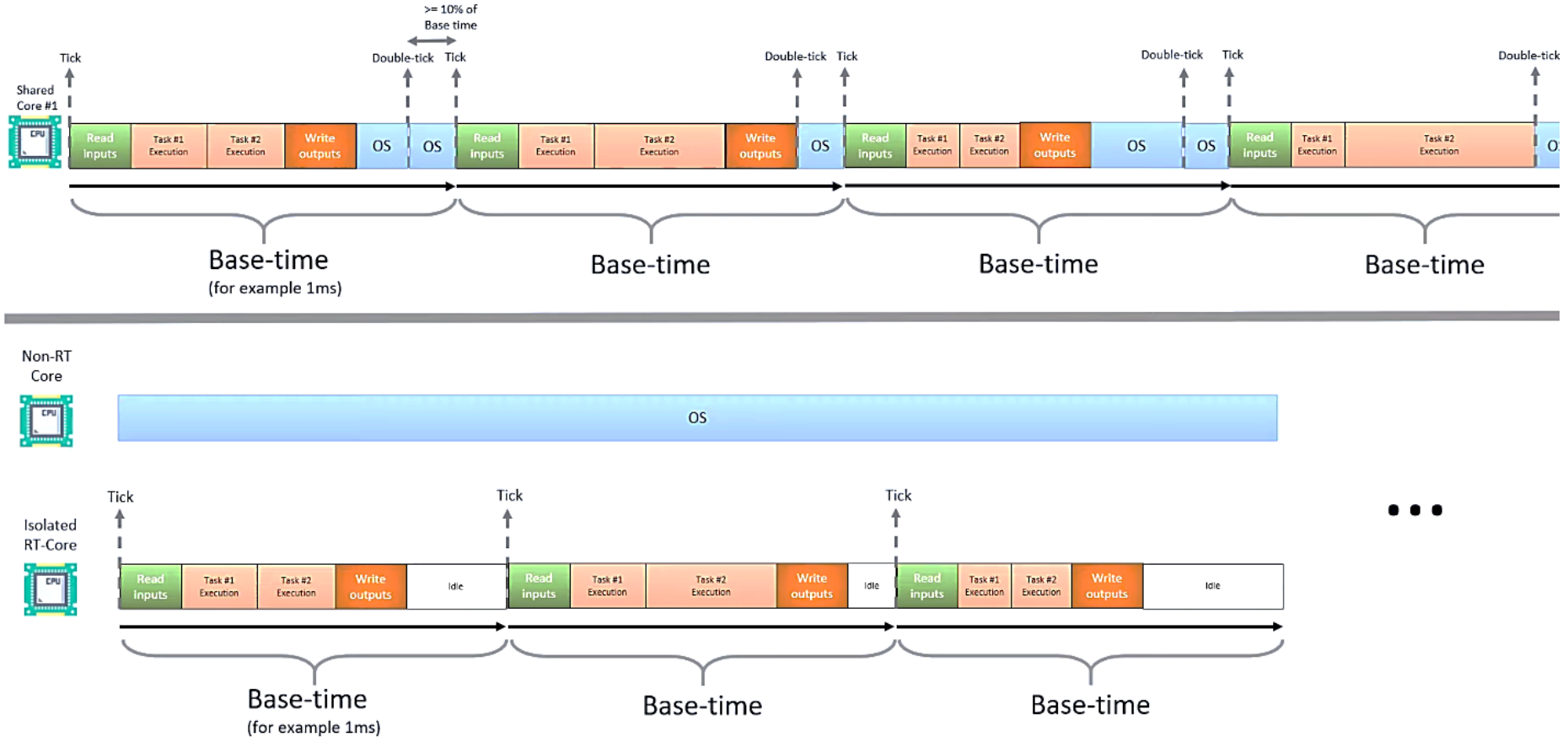
USO COMPARTIDO CPU

TwinCAT® 3

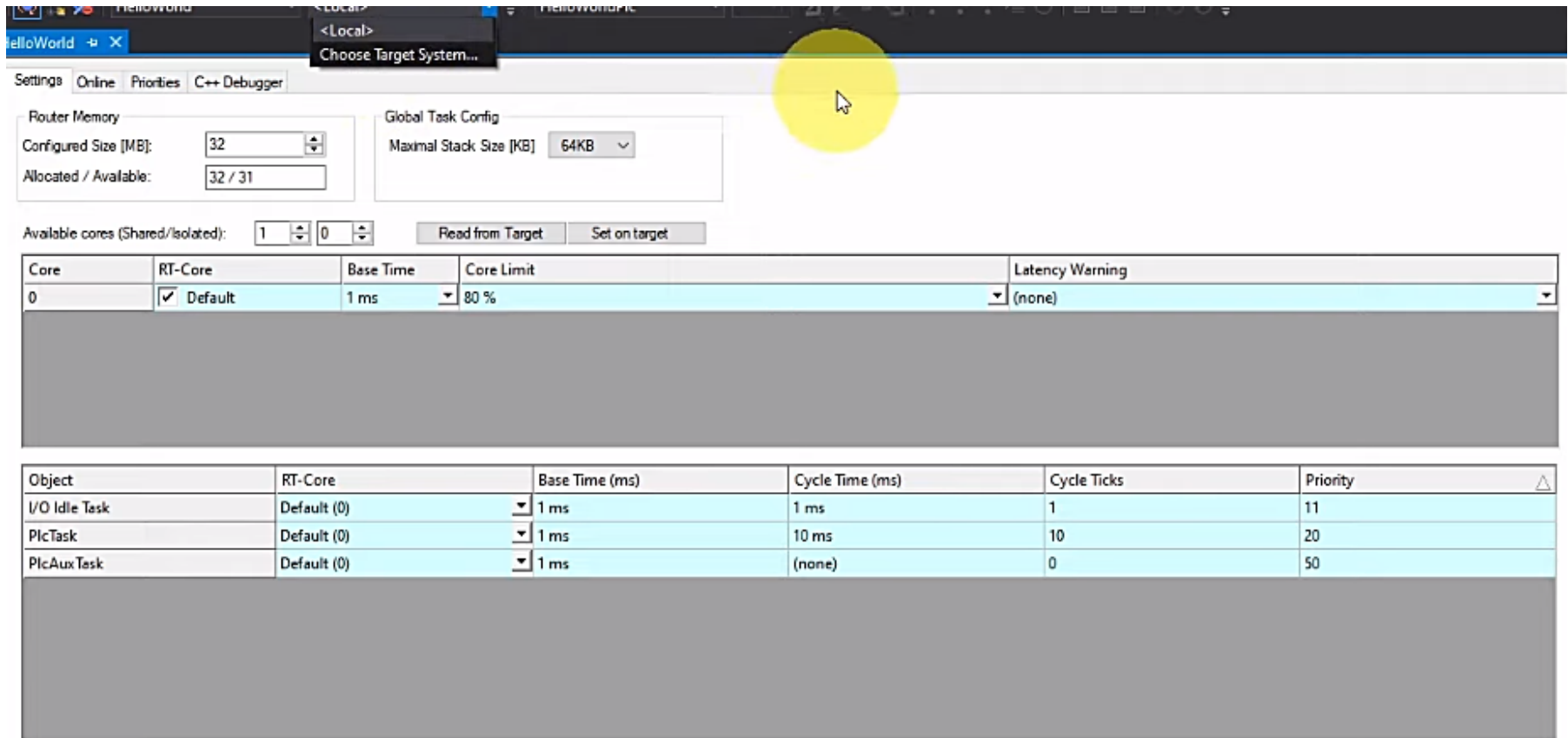


USO EXCLUSIVO CPU

TwinCAT[®] 3



DEFINICIÓN USO CPU



The screenshot shows the 'Choose Target System...' dialog in a development environment. The dialog has two main sections: 'Router Memory' and 'Global Task Config'. The 'Router Memory' section shows 'Configured Size [MB]' as 32 and 'Allocated / Available' as 32 / 31. The 'Global Task Config' section shows 'Maximal Stack Size [KB]' as 64KB. Below these sections, there are 'Available cores (Shared/Isolated)' set to 1 / 0, and buttons for 'Read from Target' and 'Set on target'. The dialog also contains two tables.

Core	RT-Core	Base Time	Core Limit	Latency Warning
0	☒ Default	1 ms	80 %	(none)

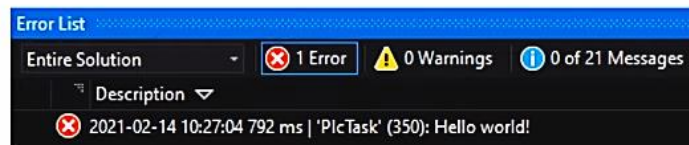
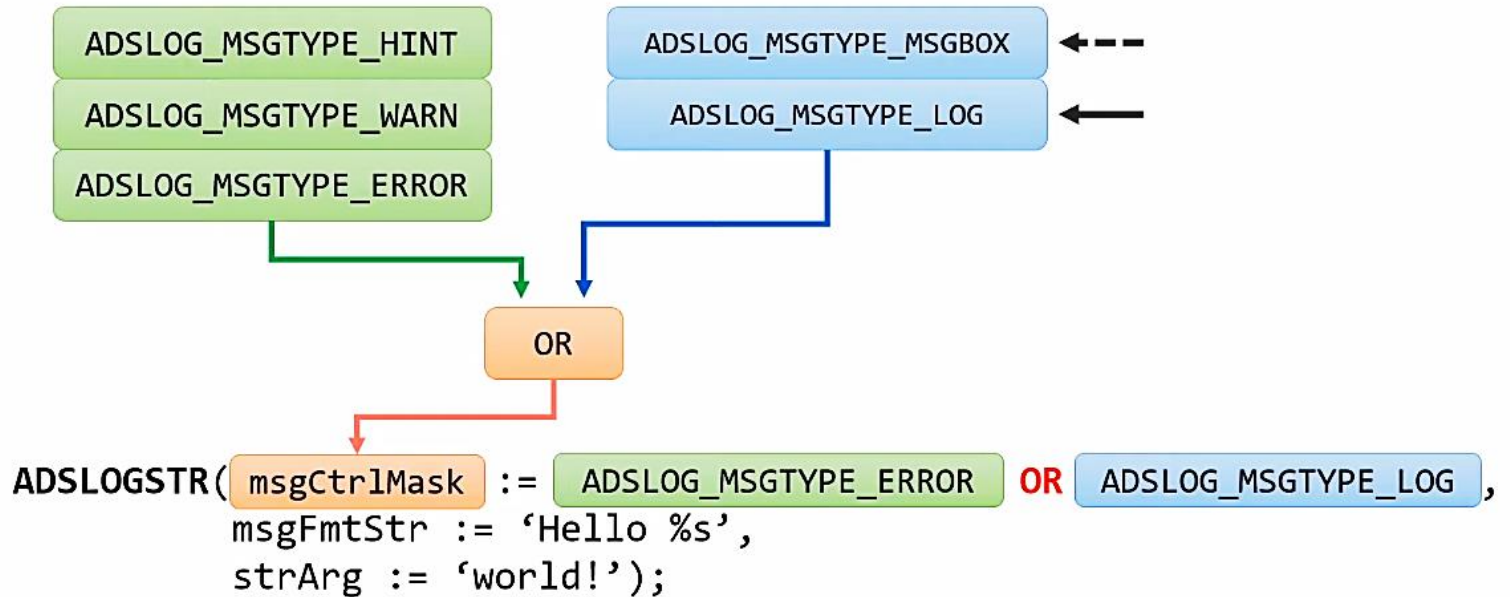
Object	RT-Core	Base Time (ms)	Cycle Time (ms)	Cycle Ticks	Priority
I/O Idle Task	Default (0)	1 ms	1 ms	1	11
PlcTask	Default (0)	1 ms	10 ms	10	20
PlcAuxTask	Default (0)	1 ms	(none)	0	50

Proyecto > SYSTEM > Real-Time > settings

ÍNDICE

- Breve historia TwinCAT
- SoftPLC
- “Hola Mundo”
- Tipos de Variable
- Tipos de Variable Compuestos
- Punteros y Referencias

ADSLOGSTR



DOCUMENTACIÓN

BECKHOFF New Automation Technology

Beckhoff Information System

[Select language] ▾

Home

Contact

www.beckhoff.com

email this page

Search

TwinCAT PLC Library: System

F_CreateAn

F_ScanAns

IO port access

MEMORY functi

Character functi

Data structures

Constants

TwinCAT PLC Lib: Math

TwinCAT PLC Lib: I/O fu

TwinCAT PLC Lib: Utiliti

TwinCAT PLC Lib: Coup

TwinCAT PLC Lib: Data

TwinCAT PLC Lib: Ether

TwinCAT PLC Lib: Drive

TS8340 | TwinCAT PLC

TS8341 | TwinCAT PLC

TS8341 | TwinCAT PLC

TS8255 | TwinCAT PLC

TS5060 | TwinCAT MC F

TwinCAT PLC Lib: MC (

TS5050 | TwinCAT MC C

TwinCAT PLC Lib: MC C

TS5055 | TwinCAT MC F

TS5065 | TwinCAT PLC

TS5065 | TwinCAT PLC

TS5066 | TwinCAT PLC

TS5810 | TwinCAT PLC

TS4110 | TwinCAT PLC

TS4100 | TwinCAT PLC

TS8010 | TwinCAT PLC

TwinCAT PLC Lib: DMX

TwinCAT PLC Lib: EnOc

TwinCAT PLC Lib: EIB

TwinCAT PLC Lib: LON

TwinCAT PLC Lib: DALI

TwinCAT PLC Lib: M-Bu

TwinCAT PLC Lib: MP-B

TwinCAT PLC Lib: SMI

TwinCAT PLC Lib: GENI

TS8000 | TwinCAT PLC

TwinCAT PLC Lib: RAID

TS6600 | TwinCAT PLC

TwinCAT PLC Lib: MDP

TS6100 | TwinCAT PLC

TS6610 | TwinCAT PLC

TS3900 | TwinCAT PLC

ADSLOGSTR

msgCtrlMask

msgFmtStr

strArg

This function issues when called a message box holding a specifiable text on the screen, and writes an entry into the system's log. In the text to be given out, a string (a sequence of characters) can be inserted in the text at a point specified by the user. For this purpose the stored format must contain the characters '%'s' at the desired location. Always remember that here too, as illustrated in the example, the function must be called using edge-control (see also the note in the description of ADSLOGDINT). The result value contains the function error code, or, if successful, 0.

FUNCTION ADSLOGSTR : DINT

```
VAR_INPUT
    msgCtrlMask : DINT;
    msgFmtStr : T_MaxString;
    strArg : T_MaxString;
END_VAR
```

msgCtrlMask : Control mask which determines the type and effect of the message output (see separate table: [ADSLOGDINT](#))

msgFmtStr : Contains the message to be issued. It can contain the formatting code '%d' for the output of a DINT value or any position altern.

strArg : Contains the string which is to be inserted into the message.

Example of calling the function in FBD:

```
graph LR
    R_TRIG[R_TRIG] --> NOT[NOT]
    bSPCError[ bSPCError ] --> AND1[AND]
    AND1 --> NOT
    NOT --> ADSLOGSTR[ADSLOGSTR]
    ADSLOGSTR --> udiAd[ udiAd ]
```

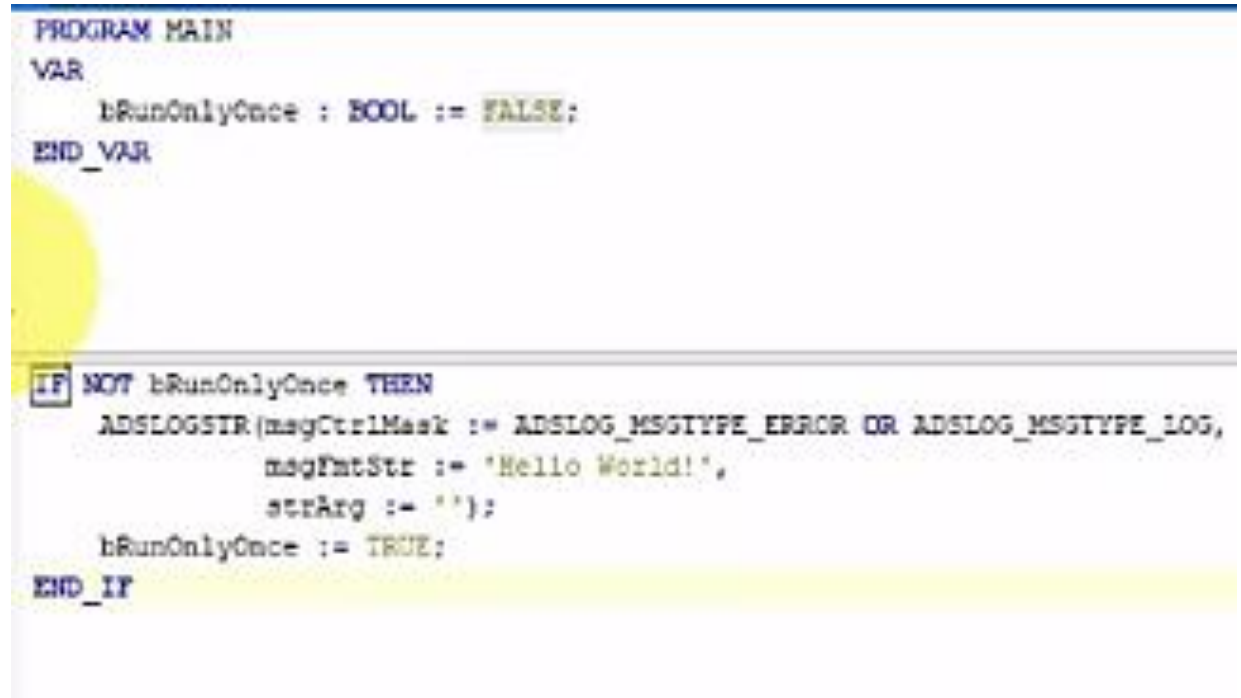
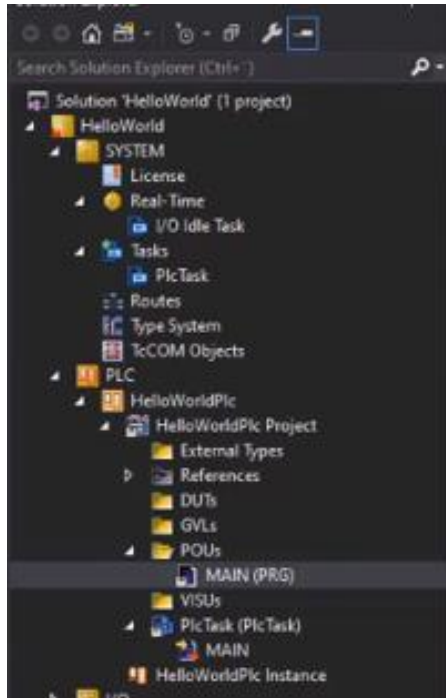
NoMessage:

```
graph LR
    AND2[AND] --> END[END]
```

https://infosys.beckhoff.com/content/1033/tcplclib_tc2_system/31033611.html&id=

25

HELLO WORLD

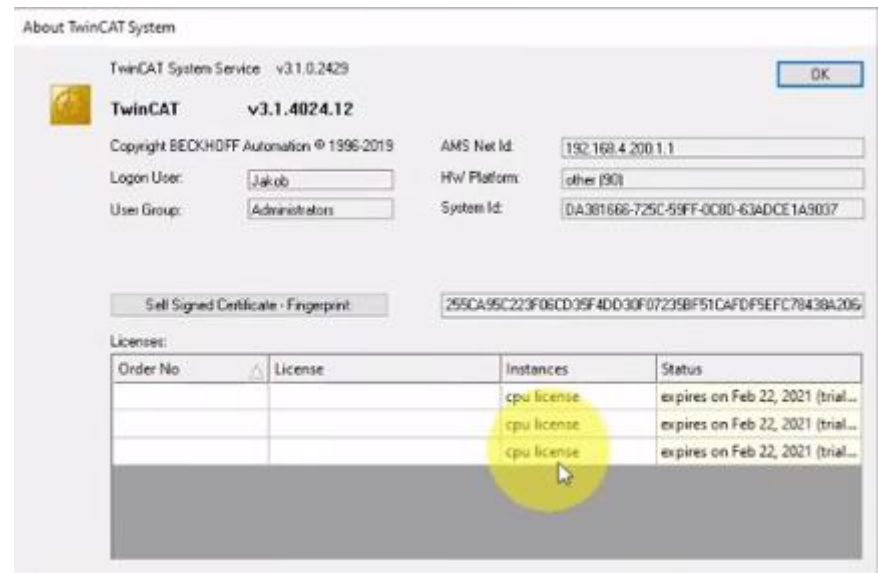


1. Crear proyecto: PLC > Add Item > “HelloWorldPLC”
2. Escribir proyecto: PLC > HelloWorldPLC > POU's > MAIN

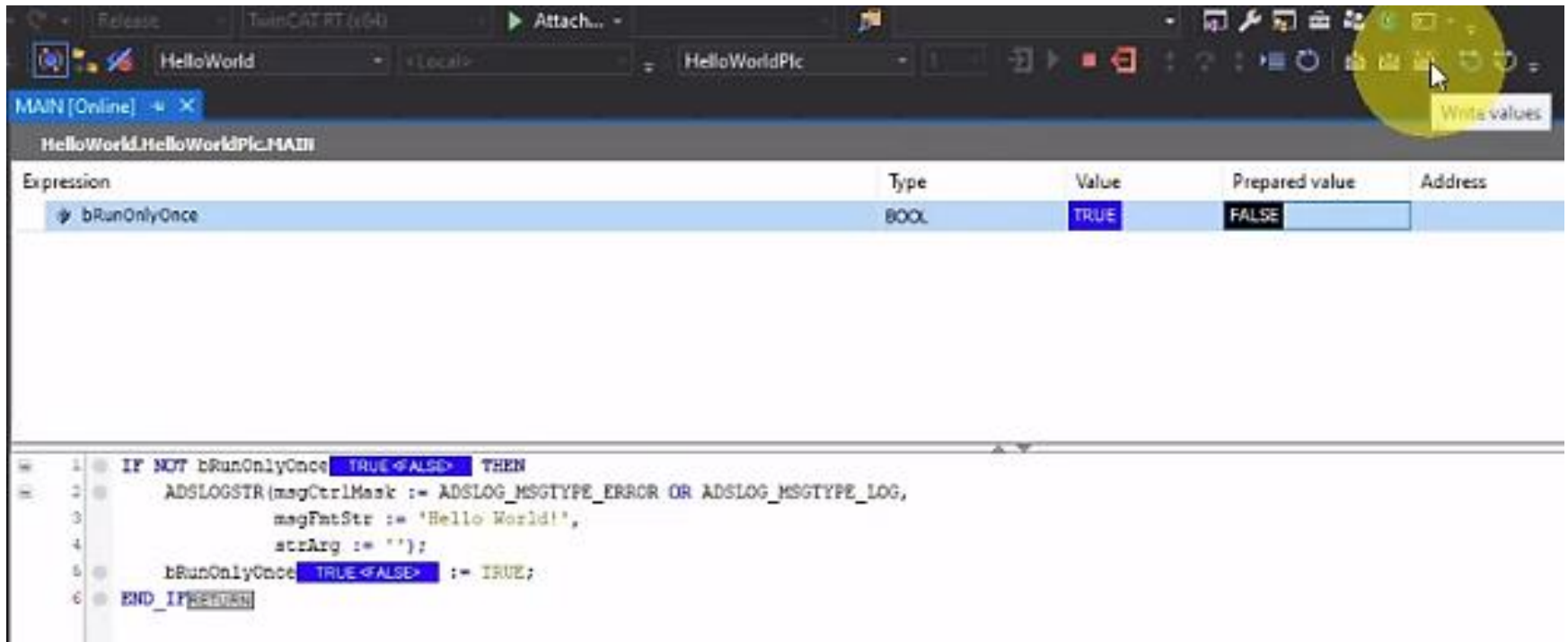
BUILD VS ACTIVATE

- BUILD: compila el programa localmente
- ACTIVATE: sube programa compilador

En caso uso personal, es necesario activar una licencia de runtime cada 15 días. La activación es gratuita.



MODIFICAR VARIABLES EN RUNTIME



The screenshot shows the TwinCAT 3 IDE interface. At the top, the project name is 'HelloWorld' and the target is 'HelloWorldPlc'. The main window displays the 'MAIN [Online]' tab, showing the variable 'bRunOnlyOnce' in the 'HelloWorld.HelloWorldPlc.MAIN' namespace. The variable is of type 'BOOL' and its current value is 'TRUE'. A yellow callout bubble with the text 'Write values' points to the 'Prepared value' field, which currently contains 'FALSE'. Below the variable table, the ladder logic code is visible, showing an 'IF NOT bRunOnlyOnce' condition that triggers a log message and sets 'bRunOnlyOnce' to 'TRUE'.

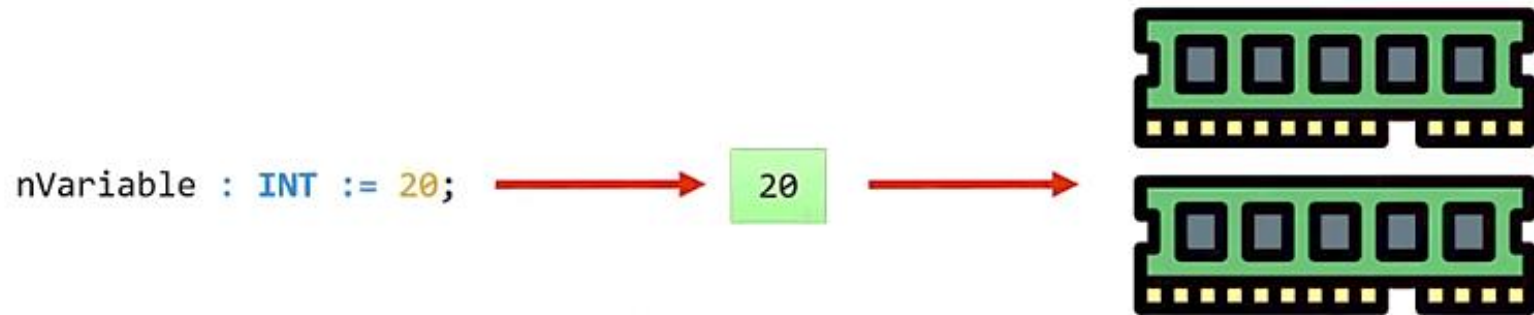
Expression	Type	Value	Prepared value	Address
bRunOnlyOnce	BOOL	TRUE	FALSE	

```
1 IF NOT bRunOnlyOnce THEN
2   ADSLOGSTR(msgCtrlMask := ADSLOG_MSGTYPE_ERROR OR ADSLOG_MSGTYPE_LOG,
3     msgFmtStr := 'Hello World!',
4     strArg := '');
5   bRunOnlyOnce := TRUE;
6 END_IF
```

ÍNDICE

- Breve historia TwinCAT
- SoftPLC
- “Hola Mundo”
- **Tipos de Variable**
- Tipos de Variable Compuestos
- Punteros y Referencias

VARIABLE



CONVENCIÓN NOMBRES

Fuente: <https://www.encyclopedia-excel.com/vba-naming-conventions-how-to-write-good-variable-names-vba-tutorial>

Snake Case	Camel Case	Pascal Case	Hungarian Notation
file_name	fileName	FileName	strFileName
current_record	currentRecord	CurrentRecord	iCurrentRecord
find_max_value	findMaxValue	FindMaxValue	fnFindMaxValue
get_user_input	getUserInput	GetUserInput	strGetUserInput
check_count	checkCount	CheckCount	dbCheckCount

recomendación CODESYS

IEC 61131-3 BOOL

TYPE	VALUES	MEMORY CONSUMPTION	EXAMPLE
BOOL	FALSE (0) TRUE (1)	8 bits (1 byte)	<code>bJustAVariable : BOOL := TRUE;</code>

IEC 61131-3 INTEGER

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
INT	-32768	32767	16 bits (2 bytes)	<code>nThisIsANumber : INT := -4232;</code>
UINT	0	65535	16 bits (2 bytes)	<code>nThisIsANumber : UINT := 44000;</code>
SINT	-128	127	8 bits (1 byte)	<code>nThisIsANumber : SINT := -3;</code>
USINT	0	255	8 bits (1 byte)	<code>nThisIsANumber : USINT := 241;</code>
DINT	-2147483648	2147483647	32 bits (4 bytes)	<code>nThisIsANumber : DINT := 1362992311;</code>
UDINT	0	4294967295	32 bits (4 bytes)	<code>nThisIsANumber : UDINT := 3453331141;</code>
LINT	-9223372036854775808	9223372036854775807	64 bits (8 bytes)	<code>nThisIsANumber : LINT := -223372036854775807;</code>
ULINT	0	18446744073709551615	64 bits (8 bytes)	<code>nThisIsANumber : ULINT := 13246311073709551615;</code>

IEC 61131-3 INTEGER

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
BYTE	0	255	8 bits (1 byte)	nSomeVariable : BYTE := 2#1011_0011;
WORD	0	65535	16 bits (2 bytes)	nSomeVariable : WORD := 16#02AE;
DWORD	0	4294967295	32 bits (4 bytes)	nSomeVariable : DWORD := 16#02AE_FFFF;
LWORD	0	18446744073709551615	64 bits (8 bytes)	nSomeVariable : LWORD := 16#18446744073709551615;

```
nSomeVariable : BYTE := 2#1011_0011;
```

```
nSomeVariable.3;
```



IEC 61131-3 FLOATING

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
REAL	-3.4×10^{38}	3.4×10^{38}	32 bits (4 bytes)	<code>fPi : REAL := 3.14159265359;</code>
LREAL	$-1.7976931348623158^{308}$	1.7976931348623158^{308}	64 bits (8 bytes)	



```
fThisIsAFloat : LREAL := 9814123441244124121244132432342342343115312341.331112571442123;
```

IEC 61131-3 STRING

TYPE	MEMORY CONSUMPTION	EXAMPLE	
STRING	80 (+1) bytes	<code>sHelloWorldString : STRING := 'Hello World!';</code> ↓ ↓	ASCII-coded
STRING(300)	300 (+1) bytes	<code>sThisIsAStringVariable : STRING(300) := 'This is a string';</code>	ASCII-coded
WSTRING	80 (+1) words 160 (+2) bytes	<code>wsHelloWorld : WSTRING := "Hello World!";</code> ↓ ↓	UNICODE-coded
WSTRING(300)	300 (+1) words 600 (+2) bytes	<code>wsJustAVariable : WSTRING(300) := "ĜHäö&â";</code>	UNICODE-coded

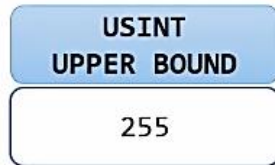
IEC 61131-3 TIME

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
TIME	0 [ms]	4294967295 [ms]	32 bits (4 bytes)	tVariable : TIME := T#1D4H5M2S132MS; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
TIME_OF_DAY (TOD)	0 (00:00:00:000) [ms]	4294967295 (23:59:59:999) [ms]	32 bits (4 bytes)	tdVariable : TIME_OF_DAY := TOD#11:14:45.577; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
DATE	0 (01.01.1970) [s]	4294967295 (7 February 2106) [s]	32 bits (4 bytes)	dDateVariable : DATE := D#1982-03-01; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
DATE_AND_TIME (DT)	0 (01.01.1970, 00:00) [s]	4294967295 (7 February 2106, 6:28:15) [s]	32 bits (4 bytes)	dtVariable : DATE_AND_TIME := DT#2013-04-18-15:31:19; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
LTIME	0 [ns]	213503d23h34m33s709ms551us615ns [ns]	64 bits (8 bytes)	ltVar : LTIME := LTIME#213503D23H34M33S709MS551US615NS; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

CONVERSIÓN ENTRE TIPOS

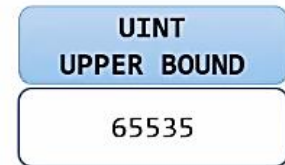
USINT

1 byte



UINT

2 bytes



```
VAR  
    nOne_Usint : USINT;  
    nOne_Uint  : UINT;  
END_VAR
```

```
nOne_Uint := nOne_Usint + 5;
```



```
nOne_Usint := nOne_Uint + 5;
```



✗ Cannot convert type 'UINT' to type 'USINT'

```
nOne_Usint := UINT_TO_USINT(nOne_Uint) + 5;
```



SIZEOF

```
MAIN  + X
1  PROGRAM MAIN
2  VAR
3      fOneFloat : REAL;
4      sThisIsAString : STRING;
5      nAnInteger : INT;
6
7      nReturnedNumberOfBytesForFloat : UDINT;
8      nReturnedNumberOfBytesForString : UDINT;
9      nReturnedNumberOfBytesForInteger : UDINT;
10 END_VAR

1  nReturnedNumberOfBytesForFloat := SIZEOF(fOneFloat);
2  nReturnedNumberOfBytesForString := SIZEOF(sThisIsAString);
3  nReturnedNumberOfBytesForInteger := SIZEOF(nAnInteger);
```

ÍNDICE

- Breve historia TwinCAT
- SoftPLC
- “Hola Mundo”
- Tipos de Variable
- **Tipos de Variable Compuestos**
- Punteros y Referencias

IEC 61131-3 ARRAYS

Alternative #1

a2dArray : **ARRAY**[1..2, 1..3] **OF** **REAL**;

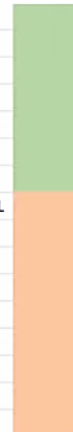
a2dArray		
2dArray[1,1]	2dArray[1,2]	2dArray[1,3]
2dArray[2,1]	2dArray[2,2]	2dArray[2,3]

Alternative #2

a2dArray2 : **ARRAY**[1..2] **OF** **ARRAY**[1..3] **REAL**;

a2dArray2		
2dArray[1][1]	2dArray[1][2]	2dArray[1][3]
2dArray[2][1]	2dArray[2][2]	2dArray[2][3]

[-] a2dArray	ARRAY [1..2, 1..3] OF REAL
[-] a2dArray[1, 1]	REAL
[-] a2dArray[1, 2]	REAL
[-] a2dArray[1, 3]	REAL
[-] a2dArray[2, 1]	REAL
[-] a2dArray[2, 2]	REAL
[-] a2dArray[2, 3]	REAL
[-] a2dArray2	ARRAY [1..2] OF ARRAY [1..3] OF REAL
[-] a2dArray2[1]	ARRAY [1..3] OF REAL
[-] a2dArray2[1][1]	REAL
[-] a2dArray2[1][2]	REAL
[-] a2dArray2[1][3]	REAL
[-] a2dArray2[2]	ARRAY [1..3] OF REAL
[-] a2dArray2[2][1]	REAL
[-] a2dArray2[2][2]	REAL
[-] a2dArray2[2][3]	REAL



IEC 61131-3 ARRAYS



VAR

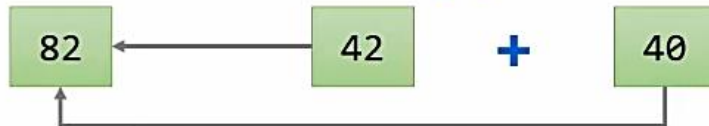
`aOneArray : ARRAY[1..5] OF INT := [42, 20, 30, 40, 50];`

`nOneInt : INT;`

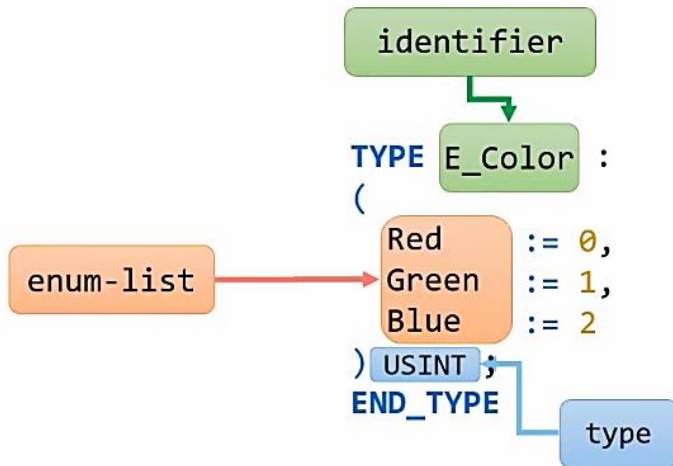
END_VAR

`aOneArray[1] := 42;`

`nOneInt := aOneArray[1] + aOneArray[4];`



IEC 61131-3 ENUM



```
VAR
    eLightTower : E_Color;
END_VAR
```

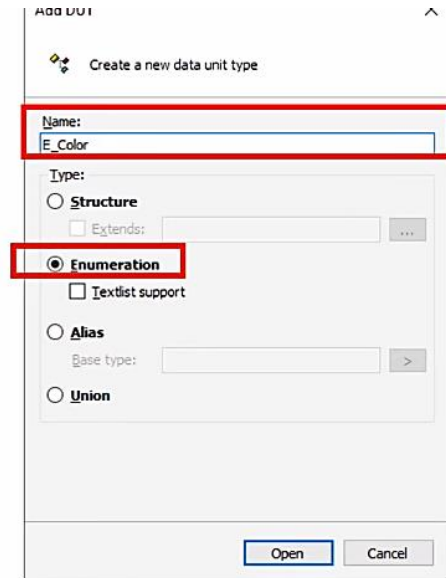
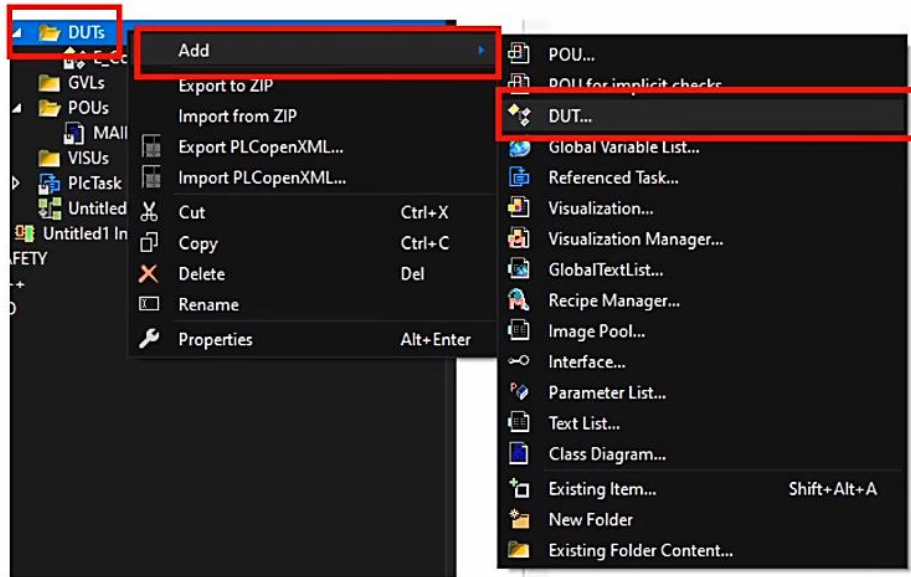
```
eLightTower := E_Color.Red;
```

```
eLightTower := E_Color.
```

A small menu box is shown next to the code, containing three options: Blue, Green, and Red, each preceded by a small icon representing a color.

Son arrays cuyos datos están etiquetados. El acceso es más significativo. El código es más fácil de depurar.

CREAR ENUM



```
E_Color
1 {attribute 'qualified_only'}
2 {attribute 'strict'}
3 TYPE E_Color :
4 (
5     enum_member := 0
6 );
7 END_TYPE
```

IEC 61131-3 ESTRUCTURAS

```
eEventType : E_EventType;  
eEventSeverity : TcEventSeverity;  
nEventIdentity : UDINT;  
sEventText : STRING(255);  
dtTimestamp : DATE_AND_TIME;
```

Permiten almacenar distintos tipos de datos en una única estructura.

IEC 61131-3 ESTRUCTURAS

```
TYPE ST_Event :  
STRUCT  
    eEventType : E_EventType;  
    eEventSeverity : TcEventSeverity;  
    nEventIdentity : UDINT;  
    sEventText : STRING(255);  
    dtTimestamp : DATE_AND_TIME;  
END_STRUCT  
END_TYPE
```

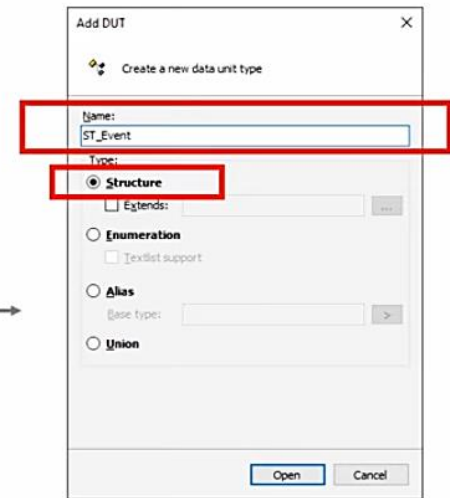
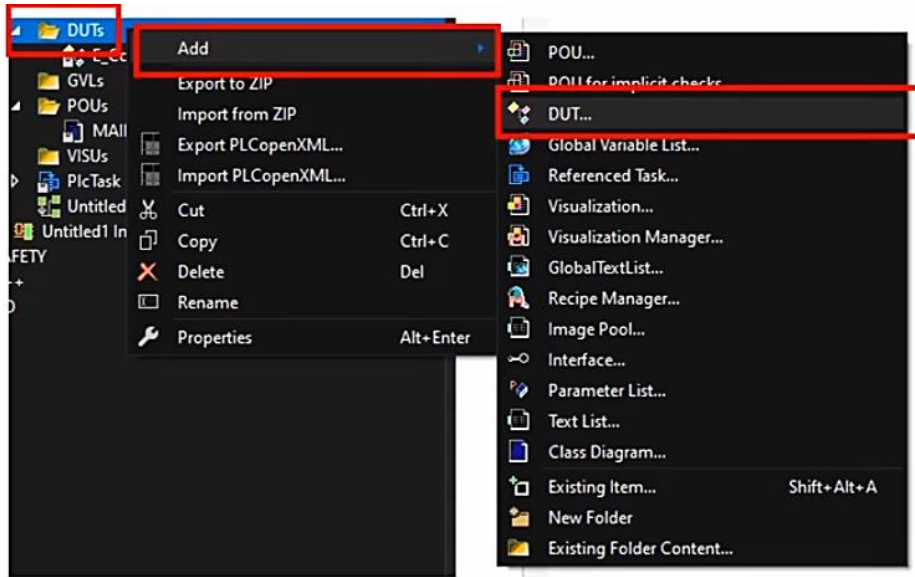
```
VAR  
    aEvents : ARRAY[1..100] OF ST_Event;  
    sTempString : STRING(255);  
END_VAR
```

```
sTempString := aEvents[33].sEventText;
```


IEC 61131-3 ESTRUCTURAS

```
aEventType : ARRAY[1..100] OF E_EventType;  
aEventSeverity : ARRAY[1..100] OF TcEventSeverity;  
aEventIdentity : ARRAY[1..100] OF UDINT;  
aEventText : ARRAY[1..100] OF STRING(255);  
aTimestamp : ARRAY[1..100] OF DATE_AND_TIME;
```

IEC 61131-3 ESTRUCTURAS



```
ST_Event  - X
1  TYPE ST_Event :
2  STRUCT
3      eEventType : E_EventType;
4      eEventSeverity : TcEventSeverity;
5      nEventIdentity : UDINT;
6      sEventText : STRING(255);
7      dtTimestamp : DATE_AND_TIME;
8  END_STRUCT
9  END_TYPE
```

ÍNDICE

- Breve historia TwinCAT
- SoftPLC
- “Hola Mundo”
- Tipos de Variable
- Tipos de Variable Compuestos
- **Punteros y Referencias**

REFERENCIAS

Pass by value

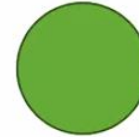


ChangeColor(



);

Pass by reference



ChangeColor(

);

La función copia la variable
para su uso interno.

REFERENCIAS

Pass by value

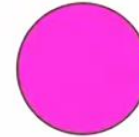


ChangeColor(

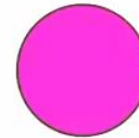


);

Pass by reference



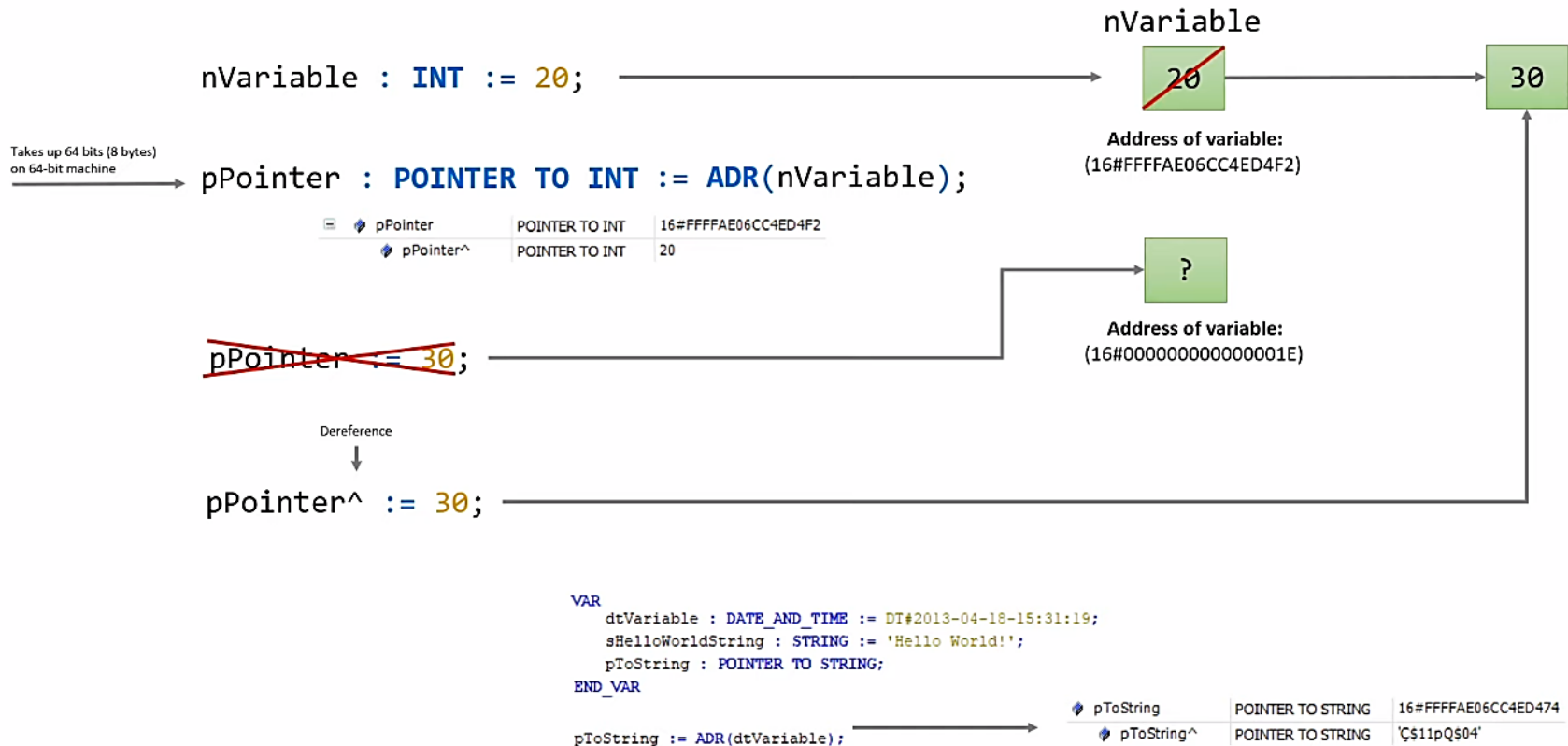
ChangeColor(



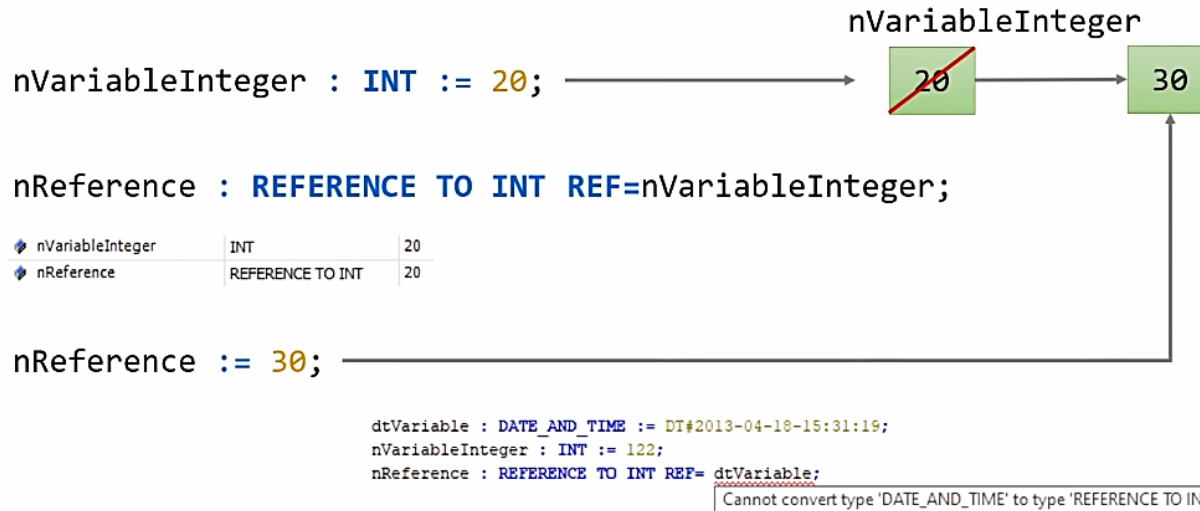
);

Usar referencias permite
ahorro de memoria

IEC 61131-3 PUNTEROS



IEC 61131-3 REFERENCIAS

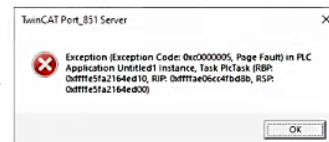


- 1 Easier to use
- 2 Type safety

IEC61131-3:

✓ Compiles

```
VAR
  nReferenceInt : REFERENCE TO INT;
END_VAR
nReferenceInt := nReferenceInt + 1;
```



ISO C++ standard (see 8.3.2/1):

"A reference shall be initialized to refer to a valid object or function"

✗ Doesn't compile

```
int& referenceToInt;
```

❗ Int-uninitialized-local Local variable is not initialized.
✗ C2530 'referenceToInt': references must be initialized

IEC 61131-3 REFERENCIAS

```
nVariable : INT := 20;
```

```
pPointer : POINTER TO INT := ADR(nVariable);
```

```
SIZEOF(pPointer) = 8
```

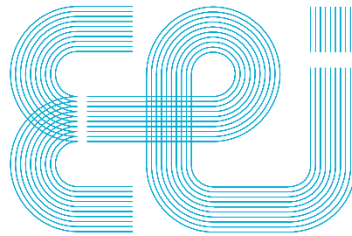
```
nVariable : INT := 20;
```

```
nReference : REFERENCE TO INT REF=nVariable;
```

```
SIZEOF(nReference) = 2
```

REGLA: Usar **referencias** siempre que se pueda. Manejo más fácil.
Ocupan menos memoria.

¿PREGUNTAS?
andsuarez@uvigo.gal



El contenido de esta presentación está basado en los materiales ofrecidos por
Jakob Satowski en <https://www.sagatowski.com/>